Verification of the score-based donation game engine

14 October 2026

1 Summary

This report lists the checks made on the engine in `engine/`, the replicator-dynamics GUI in `gui/`, and the scripts that run on them. It covers the repository as built from commit `1bb3c37` (the report and its figures are in `docs/verification/`). The notation is that of the notes *Calculating the Stable Reputation Distribution in Infinite Population Models of Indirect Reciprocity*, which were read in full (pages 1–8).

How to read it. There are 29 checks, numbered 1 to 29. Each says what was checked, what it was checked against, how many things were checked, and whether they agree. *Agree* means that no difference was above what the check allows: rounding error for an exact calculation, sampling noise for the simulation. The report does not list the tolerances. 24 checks agree, 4 agree with an exception that is explained in the check, and 1 is open (the two differ and the reason is not known).

Two things to know.

- The notes contain no ternary section, so the ternary results have no printed numbers to be compared with. Checks 10 to 19 check them against independent models, a simulation, closed forms and hand arithmetic instead.
- The open check (Check 22) is the paper's published ESS table. The engine finds the published ESS it can be expected to find but misses 44 of 403; the likely cause is the number of steps of the published run, which is not recorded.

#	Kind of check	Checked against	How many	Result
1	Binary engine	Julia code	400 cases	Agree
2	Binary ESS	Julia code	3192 rows	Agree
3	ESS sweep script	Julia code	1296 norms × 2 noise levels	Agree
4	Lifted ternary	binary engine	12 cases and 1296 norms	Agree
5	Notes: Defectors and Discriminators	the notes, pp. 5–6	7 quantities	Agree
6	Notes: ϕ-Disc rates	the notes, p. 7	5 + 1000 settings	Agree
7	Notes: ϕ-Disc, $\phi = 1$	the notes, p. 8	1 distribution, 6 settings	Agree

#	Kind of check	Checked against	How many	Result
8	Notes: $\phi = 0$	the notes	500 settings	Agree
9	Notes: stationary equations	the notes, Eq. 11	108 chains	Agree
10	Independent model: rates	independent model	5000 configurations	Agree
11	Independent model: chains	independent model	1780 chains	Agree
12	Independent model: stationary	independent model	227 + 107 settings	Agree
13	Closed form: ternary	closed form	100 configurations	Agree
14	Mismatched classes	independent model	2100 configurations	Agree
15	Edge cases	independent model	859 settings	Agree
16	Drifting flag	the rule	833 configurations	Agree*
17	Agent-based simulation	simulation, 10^6 agents	470 comparisons	Agree*
18	Hand-computed cases	hand arithmetic	18 cases	Agree
19	Planted bugs	4 test suites	25 planted bugs	Agree
20	Leading Eight	the paper's code	8 norms	Agree
21	ESS figure	the paper's code	2 figures, 20 tests	Agree
22	Published ESS data	the paper's data	403 ESS rows	Open
23	Script: Markov chain	old script, committed runs	20 + 1800 + 7	Agree*
24	Script: ESS sweep (old)	old script, Julia	1296 norms	Agree
25	Script: zeroness	old script, committed tables	20736 chains	Agree*
26	Replicator dynamics and rest points	games with known answers	26 tests	Agree
27	GUI	expected behaviour	55 unit tests, 3 browser tests	Agree
28	Saved results	the saved run	18 tests	Agree
29	Portability	Python 3.9 and 3.13	2 environments	Agree

Agree = no difference above what the check allows (rounding error for exact calculations, sampling noise for the simulation). Agree* = agree, with an exception explained in the check. Open = they differ and it is not explained.

2 The checks

Check 1: Stable scores, payoffs and rates against the Julia code

What was checked The engine's final score distributions, payoffs and the three rates α, β, γ of binary strategies and norms.

Checked against the output of `helper_funcs.jl` (saved by `make_reference.jl`).

How many 400 cases: 200 with one strategy and 200 with two, at noise 0, 0.01 and 0.1, run for 0 to 1000 steps.

Result Agree

Note Payoffs follow Julia's convention: one strategy from the distribution before the last step, two strategies from the final one.

Check 2: The ESS search against the Julia code

What was checked The list of ESS that `find_ESS` finds among the 16 binary strategies for a norm ($\rho_A = 0.99$, 100 steps, fitness threshold 10^{-5}).

Checked against the output of `helper_funcs.jl` (saved by `make_reference.jl`).

How many 3192 rows (1596 norms at noise 0 and at noise 0.01), 809 ESS in all.

Result Agree

Note Every row is compared by `tests/reference/compare_ess.py`; the test suite checks a sample of 60.

Check 3: The sweep script against the Julia code

What was checked The ESS found by the sweep script (the one that runs on Rāpoi).

Checked against The Julia ESS lists for the same norms.

How many 1296 CN-only binary norms at noise 0 and at noise 0.01.

Result Agree

Check 4: A ternary norm that ignores the cut-off against the binary result

What was checked A binary norm and binary strategies lifted to ternary classes must give exactly the binary results.

Checked against The binary engine.

How many 12 test cases (4 norms $\times$ $\tau \in \{1, 2, 5\}$), and an ESS sweep of the 1296 CN-only norms lifted to ternary.

Result Agree

Check 5: The worked example of pages 5 and 6

What was checked The stationary distributions of Defectors and Discriminators at $\epsilon = 0.1$, $\epsilon_\uparrow = 0.4$, $\epsilon_\downarrow = 0.6$, and the comparison of 2000 steps with them.

Checked against The closed-form numbers printed in the notes: $\pi_0 = 1/3$ and 0.754095, $q(\boldsymbol{\pi}_0) = 0.456286$, mean scores 0.326093 and 2, and the differences 1.10×10^{-5} and 7.36×10^{-5}.

How many 7 quantities from the notes, and both distributions at scores 0 to 20.

Result Agree

Note The notes do not say how the 2000 steps start; the initial distribution $(0.5, 0.5)$ reproduces their differences.

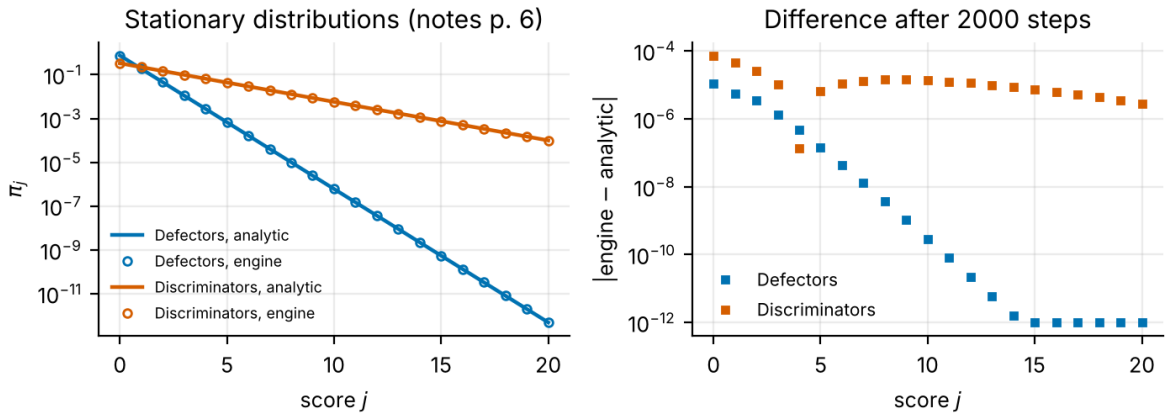


Figure 1: Defectors and Discriminators. Left: closed-form stationary probabilities (lines) and the engine after 2000 steps (circles). Right: the absolute differences.

Check 6: The ϕ-Disc rates of page 7

What was checked The up- and down-rates g_j and f_j of a population of ϕ-Disc players under `CN:00000-0+ NC:00000000`.

Checked against The formulas of the notes, $G_j = \frac{1-w_0}{4(j-1)^\phi}$ and $F_j = \frac{1-w_0}{4}(1 - j^{-\phi})$, with Eqs. 3–4.

How many 5 values of ϕ (0, 0.17, 0.5, 1, 2) at scores 1 to 7, and 1000 random settings ($\phi \in [0, 3]$, noise, score distributions).

Result Agree

Note The notes print the norm as `CN:000000-+`, which is not a valid norm (a score-0 donor would lose a score); `CN:00000-0+` is meant and is what reproduces the notes' numbers.

Check 7: The $\phi = 1$ stationary distribution of page 8

What was checked The stationary score distribution of ϕ-Disc players at $\epsilon = 0.1$, $\epsilon_\uparrow = 0.4$, $\epsilon_\downarrow = 0.6$, and its π_0 and mean score.

Checked against The gamma-function solution of the notes ($\pi_0 \approx 0.2555$, $\mathbb{E}[\text{score}] = 1.889$), with π_0 from the normalisation equation.

How many 1 distribution at all 440 tracked scores; 6 further settings of the noise.

Result Agree

Check 8: $\phi = 0$ is the Discriminators

What was checked The rates of ϕ-Disc with $\phi = 0$ and of Discriminators in the same population.

Checked against Each other (the notes say the two coincide).

How many 500 random settings of the norm, noise and score distribution.

Result Agree

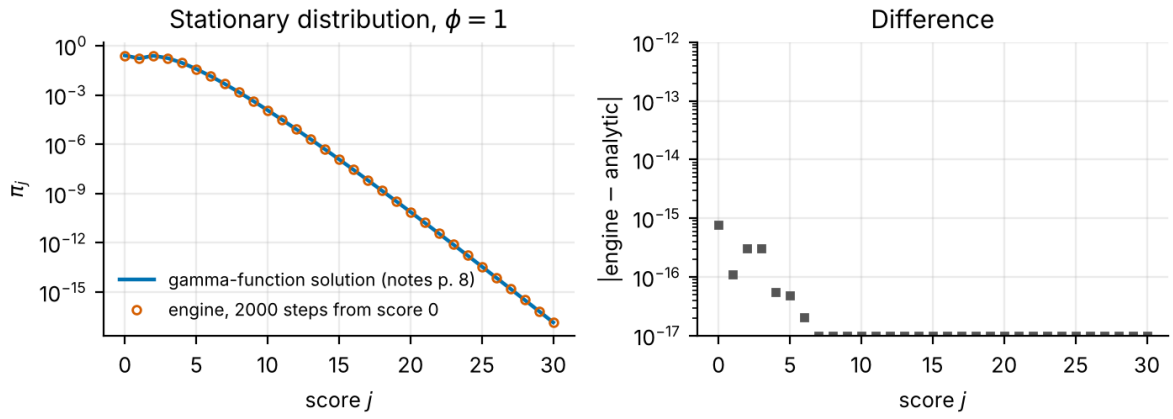


Figure 2: ϕ-Disc with $\phi = 1$. Left: the gamma-function solution (line) and the engine after 2000 steps from score 0 (circles). Right: the absolute differences.

Check 9: The fixed-point equation for π_0 (Eq. 11)

What was checked $\pi_0 = \dfrac{f - g}{f - g + g_1}$ evaluated at the engine's converged distribution.

Checked against The engine's own π_0.

How many 108 chains from 150 random binary settings with one or two strategies (the 73 settings that never settled within 4000 steps were left out).

Result **Agree**

Note The detailed-balance equations (Eqs. 5–6) are checked on the stationary distributions in Check 12.

Check 10: The rates G, F, g, f against an independent implementation

What was checked The transition rates for random populations of binary, ternary and ϕ-Disc strategies.

Checked against an independent re-implementation of the model written from the notes and `helper_funcs.jl` without looking at the engine's code (`tests/independent_model.py`).

How many 5000 random configurations: binary, ternary and lifted norms; $\tau \in \{1, 2, 3, 5, 8\}$; one to three strategies of mixed kinds (probabilistic and deterministic tables, $\phi \in [0, 3]$); noise from 0 to 1; shares of zero; score distributions with mass beyond τ.

Result **Agree**

Check 11: The score chains over time against the independent implementation

What was checked The score distributions after 25 to 400 steps of the chain, with mass conserved.

Checked against The independent implementation iterating the same chain.

How many 1780 chains.

Result **Agree**

Check 12: Stationary distributions

What was checked Whether a stationary distribution found independently (Eqs. 5–6 and a fixed point on the class masses) is left unchanged by one engine step, and whether the engine's own converged runs reach it.

Checked against The independent implementation.

How many 227 random settings for the first, 107 for the second.

Result **Agree**

Note How close a converged run gets is set by the engine's stopping rule, not by the model.

Check 13: Three geometric segments for a class-based ternary strategy

What was checked The stationary distribution of one ternary strategy, which is geometric over the three classes.

Checked against The closed form, with the class masses solved self-consistently, independently of the engine.

How many 100 configurations ($\tau \in \{1, 2, 3, 5, 8\}$).

Result **Agree**

Check 14: Strategies whose classes differ from the norm's

What was checked Rates when a strategy's cut-off τ differs from the norm's, or a binary strategy meets a ternary norm and the other way round (the engine uses the raw scores).

Checked against The independent implementation.

How many 2100 configurations.

Result **Agree**

Check 15: Awkward settings

What was checked Rates and chains at $\tau = 1$, with an empty top class, with $\epsilon = 0$, and with a strategy whose share is 0.

Checked against The independent implementation (a population with and without the strategy of share 0).

How many $\tau = 1$: 300; empty top class: 200; $\epsilon = 0$: 159; share 0: 200.

Result **Agree**

Check 16: Which chains are flagged as drifting

What was checked That the engine flags a chain exactly when its up-rate at the top score is positive and at least its down-rate.

Checked against The rule, evaluated independently.

How many 833 configurations.

Result **Agree** (with the exception below)

Note When nothing can move (both rates 0) the engine does not flag the chain; the README now says so.

Check 17: A simulated population against the engine

What was checked Score histograms after the same number of steps: a population of 10^6 agents meeting random partners, with roles, help, the norm and noise drawn as in the model, against the engine.

Checked against The engine, bin by bin, with binomial standard errors.

How many 470 comparisons (the two example runs, ϕ-Disc alone, Defectors against Discriminators, random ternary norms, a mix of ϕ-Disc and a ternary strategy); 11 stationary scenarios, 48 comparisons.

Result **Agree** (with the exception below)

Note Two comparisons, both in one random case that the engine flags as drifting, fall outside the noise: the default stopping tolerance leaves the engine short of the stationary law there. The same test detects a 10% error in ϵ and a 4% error in the role factor.

Check 18: Small cases worked out on paper

What was checked Rates for tiny populations at $\tau = 1$ and 2: CN against NC, the recipient's index against the donor's, the class boundary at scores τ and $\tau + 1$, the noise arithmetic, the ϕ-Disc exponent, one step of the chain.

Checked against Numbers written as literal arithmetic from the notes.

How many 18 cases.

Result **Agree**

Check 19: Would the checks notice a mistake?

What was checked Whether the test suites fail when a bug is planted in a copy of the engine (a wrong cut-off, a swapped CN and NC, a transposed table, a wrong lift to ternary, the wrong score in the ϕ exponent, a wrong role factor, ...).

Checked against Four suites: the hand cases, the independent tests, the repository's own tests and the simulation tests.

How many 25 planted bugs.

Result **Agree**

Note Every bug was caught by at least one suite. No single suite caught them all (the repository's own tests missed three cut-off bugs; the simulation tests missed a transposed table and a wrong lift), which is why all four are kept.

Check 20: The Leading Eight norms

What was checked The barcodes, names and ESS strategies of L1 to L8 in `known_norms.py`, and whether the engine finds a fully cooperative ESS for each.

Checked against `createLeading8()` in the paper's code, and the engine's own ESS search (noise 0.001 and 0.01).

How many 8 norms: barcodes and strategies equal; a cooperative ESS found for 8 of 8.

Result **Agree**

Note The five entries all eight share and the three that vary are also checked in the tests.

Check 21: The ESS figure against the paper's

What was checked `plots/ess_plot.py`: the layout, sort order, cell widths, colours and Leading Eight labelling.

Checked against The paper's `plotting_funcs.py` and the figures it produced.

How many 2 data files drawn and compared by eye (25 and 403 rows), and 20 unit tests.

Result **Agree**

Check 22: The paper's published ESS table

What was checked The ESS the engine finds for every CN-only norm at noise 0.01 and 500 steps.

Checked against The paper's file `tokens_CmeetsN_Noise0.01_B2`.

How many 403 published ESS rows (302 norms).

Result **Open** (not yet explained)

Note The engine finds 359 of them; every ESS it finds with fitness above 0.01 is in the file. 44 published ESS are not found, and the fitness values of the common rows differ by about 5×10^{-4}. Re-running the 65 norms where they differed with 200 steps leaves 20 of 113 published ESS missing. The number of steps of the published run is not recorded; fewer steps would explain missing ESS whose chains have not settled.

Check 23: `run_markov_chain.py` on the engine

What was checked Rates, class masses and score distributions of every step, and the number of steps.

Checked against The old script and the seven run directories committed in the repository.

How many 20 argument sets, 1800 random norms, strategies and initial distributions, and 7 committed runs.

Result **Agree** (with the exception below)

Note The old script capped scores at `maxw`; the engine does not. Where the cap did not bind the two agree. Where it did, the rates and class masses still agree in the committed runs while the score distributions continue above the cap; with a very small cap the old results differ.

Check 24: `experiments/ESS/find_ESS_parellel.py` on the engine

What was checked The ESS found for every CN-only norm, and the CSV written.

Checked against The old code, the committed CSV and the Julia ESS lists.

How many 1296 norms (333 ESS rows); the ternary path in 64 cases.

Result **Agree**

Note The old code took two-strategy payoffs from the distribution before the last step and the engine, like Julia, from the final one; this only reorders near-ties and changes no ESS.

Check 25: `experiments/verify-zeroness-one-strategy` on the engine

What was checked The group of initial rates, the final rates and the share of score 0 of every chain, and the limit-check tables.

Checked against The old code and the committed tables.

How many 20736 chains (1296 norms × 16 strategies); the 20736-row limit-check table; the rate function at 11 class masses for every chain.

Result **Agree** (with the exception below)

Note The old chains were capped at `maxw`, so about half of them now track more scores; the limit check keeps its own capped chain.

Check 26: Games with known answers

What was checked Rest points and their stability, the replicator equation, and the restart of the score chains.

Checked against Games whose answers are known (prisoner's dilemma, hawk–dove, coordination with two and three strategies, rock–paper–scissors in both versions), the invasion fitness of the engine's own payoffs, and the analytic Cooperators against Defectors.

How many 26 tests (19 for the simplex and rest points, 7 for the dynamics).

Result **Agree**

Check 27: The GUI logic and a click-through in a browser

What was checked Input checks, settings files, presets, figures, the strategies and norm cards, and a click through all three pages in Chromium (including going back to the results, typing axis ranges and saving).

Checked against The expected behaviour.

How many 55 unit tests, and 3 browser tests (full click-through, two strategies from a settings file, save and load).

Result **Agree**

Check 28: Saving a result and loading it again

What was checked That a loaded result has every number of the saved one, that a new point can still be computed, that the file is plain queryable tables, and that a taken name, a damaged file and a foreign file are refused.

Checked against The finished run that was saved (2 and 3 strategies).

How many 18 tests; every point on the simplex, every trajectory step, every matrix and distribution.

Result **Agree**

Note The far tail of each score distribution (probabilities below 10^{-12}) is not stored and reads back as 0.

Check 29: Other versions of Python

What was checked That the engine and the sweep run on the Python that Rāpoi provides.

Checked against Python 3.9 with numpy 1.26 and scipy 1.11, and Python 3.13 with numpy 2.5 and scipy 1.18.

How many 2 environments: 517 engine tests and the sweep tests on 3.9; the whole suite on 3.13.

Result Agree

3 Tests in the repository

`python -m pytest` runs 669 tests, all passing; 3 browser tests (`-m e2e`, Chromium through Playwright) are run separately and also pass. Many of the checks above are in the suite, so they run every time.

File	Tests	Covers
`test_against_julia.py`	460	400 stable-score cases, 60 sampled ESS rows
`test_engine.py`	32	norms, strategies, closed forms, invariants
`test_independent_model.py`	19	independent re-implementation
`test_agent_based.py`	12	agent-based simulation
`test_hand_cases.py`	1	18 hand-computed cases
`test_simplex.py`	19	grid, rest points, stability
`test_replicator.py`	7	dynamics, restart, role factor
`test_known_norms.py`	12	Leading Eight
`test_plots.py`	20	ESS figure and readers
`test_saves.py`	18	saving and loading results
`test_gui_logic.py`	55	GUI logic
`test_ess_sweep.py`	6	ESS sweep script
`test_experiments.py`	4	migrated experiment scripts
`test_run_markov_chain.py`	4	Markov chain script
Total	669	

4 What has not been verified

- The conventions for ternary strategies and norms (the cut-off at τ, the order of the 18 characters of a barcode, the orientation of a strategy table) are not printed in the notes. They rest on the description given for the engine and on the lifting identity (Check 4); the independent models share them with the engine.
- The notes give a closed form for ϕ-Disc only at $\phi = 1$. For other ϕ the engine agrees with the rates in the notes, but no stationary distribution has been compared with a closed form.

- The role factor $q(1-q)$ is tested against its definition and by scaling, but Julia only ever uses $q = 1/2$.
- The replicator dynamics, the rest-point classification and the GUI have no reference implementation; they rest on games with known answers and on the tests above.
- Results at the default tolerance (10^{-5} on the change in one step) are approximate for slowly relaxing chains, in the engine and in the Julia code alike. A chain flagged as drifting has no stable distribution at the rates it has at the top score, but one can exist for other class masses, and the chain can take very long to reach it.
- Payoffs and the ESS search were checked against Julia for binary strategies only; for ternary strategies the independent checks cover rates, chains and stationary distributions, not the ESS search.
- The Slurm scripts in `scripts/raapoi/` have been tested locally, with the Slurm environment variables set by hand, but have not been run on Rāpoi itself.

5 Reproducing the checks

```
python -m pytest                            # the tests above
python -m pytest -m e2e                     # browser tests (needs playwright)
python tests/reference/compare_ess.py       # every ESS row against Julia
PYTHONPATH=. python docs/verification/make_figures.py   # the two figures
python docs/verification/build_report.py    # the checks of this report
```